01 · Explore the dataset

Transcript Intelligence: notebook 1 of 5

This notebook profiles the call transcripts, checks data quality, and explains the fields that `src/enrich.py` derives because the raw data doesn't have them (call type, account, products, period and team). It ends with a first look at the provided topics and sentiment, which shapes the approach in the topic and sentiment notebooks.

Notebook	Question it answers
01_explore (this one)	What's in the data, is it trustworthy, and how is it structured?
02_topics	What are the calls about? (Task 1)
03_sentiment	How do people feel, and how is that changing? (Task 2)
04_insights	What else should each stakeholder know? (Task 3)
05_dashboard	How does the dashboard (`app/app.py`) present it all?

How to run. Run the pipeline first from the project root (`python -m src.pipeline`), then **Restart** → **Run All** here. This notebook only reads the saved tables in `data/processed/` . It doesn't call the LLM and costs nothing to re-run.

1. Setup

In [1]:
```
%load_ext autoreload
%autoreload 2

import sys
from pathlib import Path

ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
sys.path.insert(0, str(ROOT))
```

```
import pandas as pd
import plotly.express as px
import plotly.io as pio

from src.load import RAW_DIR, PROCESSED_DIR
from src.enrich import load_enriched, OUTAGE_START, OUTAGE_END, COMPLY_LAUNCH, PERIODS

pio.renderers.default = "plotly_mimetype+notebook_connected"   # shows in VS Code and in HTML exports
pd.set_option("display.max_columns", 50)
pd.set_option("display.max_colwidth", 110)

TYPE_COLORS = {"external": "#2a78d6", "support": "#eb6834", "internal": "#8a8f98"}
print("Project root:", ROOT)
```

```
Project root: C:\dev\transcript_intel
```

2. Load the tables

`src/load.py` flattens each meeting folder into four tables. `src/screen.py` then drops non-business calls such as birthdays and farewells (section 4), and `src/enrich.py` adds derived fields and builds a staff roster. Every table shares `call_id`, so they join cleanly. Everything in this notebook is **business calls only**.

Decision: mark synthetic calls. The assignment's dataset covers 3 Feb – 28 Apr 2026. Calls dated after that are synthetic transcripts I generated to test new input and continue the dataset's storylines (see `C:\dev\New_Input\README.md`). They're flagged with `is_synthetic`. Headline statistics use the **original 100** so they aren't skewed by data I wrote myself.

In [2]:
```
t = load_enriched(PROCESSED_DIR)
calls, sents, moments, actions, roster = (t[k] for k in
    ["calls", "sentences", "key_moments", "action_items", "staff_roster"])

ORIGINAL_END = pd.Timestamp("2026-04-28").date()
calls["is_synthetic"] = calls["date"].map(lambda d: pd.Timestamp(d).date() > ORIGINAL_END)
orig = calls[~calls["is_synthetic"]].copy()

pd.DataFrame({
    "rows": {k: len(v) for k, v in t.items()},
    "grain": {"calls": "1 per meeting", "sentences": "1 per sentence", "key_moments": "1 per tagged moment",
```

```
                "action_items": "1 per action item", "staff_roster": "1 per AegisCloud employee"},
})
```

Out[2]:

	rows	grain
calls	115	1 per meeting
sentences	4778	1 per sentence
key_moments	490	1 per tagged moment
action_items	466	1 per action item
staff_roster	27	1 per AegisCloud employee

In [3]:
```
print(f"Calls: {len(calls)} total = {len(orig)} original + {calls['is_synthetic'].sum()} synthetic")
print(f"Original period: {orig['date'].min()} → {orig['date'].max()}"
      f"  ({(pd.Timestamp(orig['date'].max()) - pd.Timestamp(orig['date'].min())).days + 1} days)")
```

```
Calls: 115 total = 100 original + 15 synthetic
Original period: 2026-02-03 → 2026-04-28  (85 days)
```

What one raw meeting looks like

Each meeting is a folder of six JSON files. Only `meeting-info`, `summary` and `transcript` are required by the loader. A folder missing one of them is logged and skipped, so one bad file can't stop the run.

In [4]:
```
sample = sorted(p for p in RAW_DIR.iterdir() if p.is_dir())[0]
pd.DataFrame([{"file": f.name, "size_kb": round(f.stat().st_size / 1024, 1)} for f in sorted(sample.iterdir())])
```

Out[4]:

	file	size_kb
0	events.json	0.9
1	meeting-info.json	0.6
2	speaker-meta.json	0.1
3	speakers.json	3.6
4	summary.json	3.0
5	transcript.json	13.0

3. Data quality

Before analysing anything, check whether the data can be trusted: are values missing, can every sentence be tied to a person, and how complete are the transcripts?

In [5]:

```
checks = {
    "calls with no summary":            (calls["summary"].str.len() == 0).sum(),
    "calls with no sentiment score":    calls["sentiment_score"].isna().sum(),
    "calls with no transcript text":    (calls["n_sentences"] == 0).sum(),
    "duplicate call_ids":               calls["call_id"].duplicated().sum(),
    "sentences matched to an email":    f"{sents['speaker_email'].notna().mean():.1%}",
    "sentences with speaker side known": f"{(sents['speaker_side'] != 'unknown').mean():.1%}",
    "median ASR confidence":            round(sents["confidence"].median(), 2),
    "lowest ASR confidence":            sents["confidence"].min(),
}
pd.Series(checks, name="value").to_frame()
```

Out[5]:

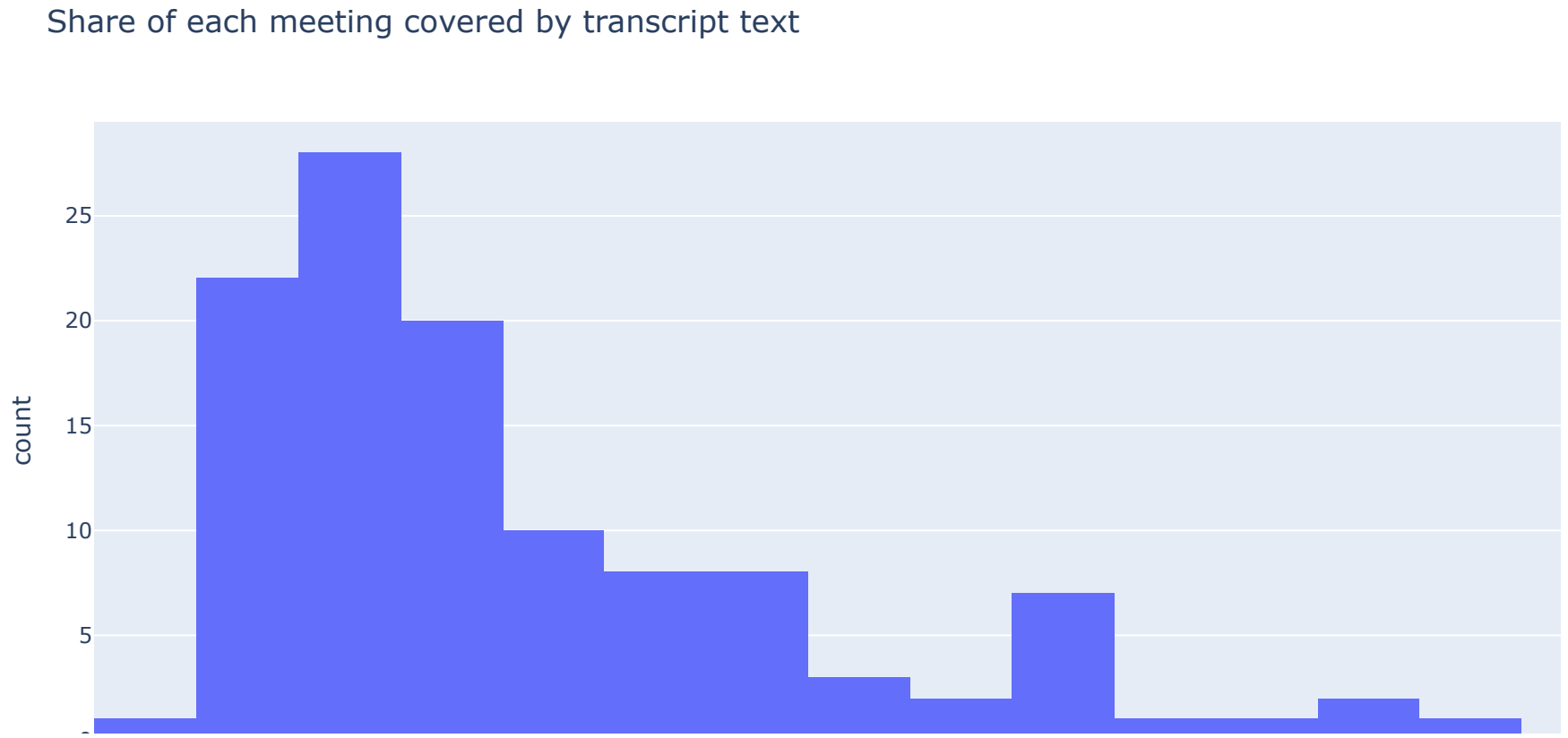
	value
calls with no summary	0
calls with no sentiment score	0
calls with no transcript text	0
duplicate call_ids	0
sentences matched to an email	100.0%
sentences with speaker side known	100.0%
median ASR confidence	0.92
lowest ASR confidence	0.88

Transcripts cover only part of each meeting

Comparing the last transcribed second with the meeting's duration shows that transcripts are **partial**. This matters for anything time-based: talk-time share is a share of the *transcribed* portion, not of the whole meeting.

In [6]:
```
last_word = sents.groupby("call_id")["end_s"].max()
coverage = (last_word / (calls.set_index("call_id")["duration_min"] * 60)).rename("coverage")
print(coverage.describe(percentiles=[.25, .5, .75]).round(2).to_string())
px.histogram(coverage, nbins=20, title="Share of each meeting covered by transcript text",
             labels={"value": "transcript end ÷ meeting duration"}).update_layout(showlegend=False)
```

```
count    115.00
mean       0.34
std        0.15
min        0.15
25%        0.24
50%        0.29
75%        0.41
max        0.90
```



What this shows. The data is clean structurally: no missing summaries or scores, every sentence maps to a named attendee, and ASR confidence is high (0.88–0.97). The main caveat is coverage. Transcripts typically capture about a third of the meeting, so the analysis treats them as samples of the conversation rather than complete records.

4. Non-business calls: screened out before analysis

Not every recorded meeting is about work. A birthday, farewell or happy-hour call dropped into `data/raw/` would be loaded like any other call and quietly distort everything after it:

- a very positive internal call lifts confidence for its month and period;
- its "action items" (order the cake) count as workload, and its praise moments as customer praise;
- a call dated weeks after the last real one stretches the time range;
- if a customer is on it, it becomes that account's **latest call** and can make an at-risk account look healthy.

Decision: screen every call right after loading (`src/screen.py`), before any field is derived, so no table, notebook, CSV export or dashboard view can count a non-business call by accident.

Check	How	When it decides
Keyword rule	`NON_BUSINESS` (birthday, celebration, happy hour, farewell, holiday party, potluck, ...) on the **title and vendor topics only**, because summaries say things like "celebrated the renewal"	Rules-only runs, or when Claude gives no answer. It also flags new transcripts in the dashboard before they're processed
`is_business` (Claude)	Title, attendee mix (staff count, customer domains), summary and vendor topics → `{is_business, reason}` . A work meeting that ends with a celebration is still business. Cached per call, about $0.002 each	Hybrid runs: Claude's answer wins
Ignore (dashboard)	A person excludes a new transcript before it's processed	Always

Every excluded call is recorded in `data/exclusions.csv` (who decided, why, when), and its folder is **moved** from `data/raw/` to `data/exclude/` . Setting its `action` to `keep` moves it back on the next run and stops it being screened out again. This notebook only reads cached answers; it never calls the API.

In [7]:
```
from src.screen import (NON_BUSINESS, SYSTEM_PROMPT as SCREEN_PROMPT, EXCLUSIONS, EXCLUDE_DIR,
                        keyword_hit, cached_screen, load_exclusions)
from src.load import load_tables
from src.llm import load_usage

print("Keyword rule (title + vendor topics):", NON_BUSINESS.pattern, sep="\n")
examples = ["Team Birthday Celebration - Priya", "Friday Happy Hour", "Farewell lunch for Sam",
            "Aegis / Summit Trust - Renewal Decision", "Comply v2 - Launch Day Checklist", "All Hands - April Update"]
pd.DataFrame({"title": examples, "keyword hit": [keyword_hit(t) or "—" for t in examples]})
```

```
Keyword rule (title + vendor topics):
\b(?:birthday|b-?day|celebrations?|happy hour|farewell|going[- ]away|baby shower|bridal shower|potluck|holiday party|christmas
party|office party|team (?:lunch|dinner|outing|social)|game night|trivia night|social hour|retirement party|welcome (?:lunch|pa
rty)|cake)\b
```

Out[7]:

	title	keyword hit
0	Team Birthday Celebration - Priya	Birthday
1	Friday Happy Hour	Happy Hour
2	Farewell lunch for Sam	Farewell
3	Aegis / Summit Trust - Renewal Decision	—
4	Comply v2 - Launch Day Checklist	—
5	All Hands - April Update	—

In [8]:
```
# The instructions Claude gets for the is_business answer
print(SCREEN_PROMPT)
```

```
You screen recorded calls at AegisCloud, a B2B cloud-security company (products: Detect, Protect, Comply, Identity, LogVault),
before they go into a business-analytics dashboard.

is_business is true if any substantive part of the call is work: customers, sales, renewals, support cases, incidents, product
s, engineering, planning, operations, hiring or internal strategy. It is false for social or personal calls with no work conten
t: birthdays, celebrations, farewells, happy hours, holiday parties, casual chats. A work meeting that starts or ends with a ce
lebration is still business.
reason is one short sentence citing what in the call decided it.
```

In [9]:
```
# Every loaded call, as screened (cached answers only; no API calls)
screened = cached_screen(load_tables(PROCESSED_DIR)["calls"])
print(f"{len(screened)} loaded calls: {int(screened['is_business'].sum())} business, "
      f"{int((~screened['is_business']).sum())} not; decided by {screened['decided_by'].value_counts().to_dict()}; "
      f"keyword hits among them: {int(screened['keyword'].notna().sum())}")
# Claude's reasoning on the internal meetings closest to the line
screened[screened["title"].str.contains("All Hands|At-Risk|Game Day|Executive", case=False)][["title", "claude_reason"]]
```

```
115 loaded calls: 115 business, 0 not; decided by {'llm': 115}; keyword hits among them: 0
```

Out[9]:

	title	claude_reason
24	All Hands - February Wrap	The call covered Comply v2 GA timing, QA coverage, a customer loss to VaultEdge, ~$240K ARR in upsells, a ...
95	All Hands - April Update	The all-hands covered the Detect outage post-mortem, the Comply v2 launch status and bugs, and Q2 roadmap ...
102	Detect Chaos Engineering - Game Day Review	The call is an engineering review of a Detect chaos engineering game day, covering failover results, a mon...
107	Customer Success - At-Risk Account Review	The call is an internal Customer Success review of at-risk accounts covering renewals, churn to a competit...
113	Q2 Executive Business Review	The call is a Q2 executive business review covering net revenue retention, customer churn and renewals, in...

In [10]:
```
ex = load_exclusions()
print(f"Exclusion list: {EXCLUSIONS} ({len(ex)} row(s))")
print("Folders in data/exclude/:", sorted(p.name for p in EXCLUDE_DIR.iterdir()) if EXCLUDE_DIR.exists() else "none")
usage = load_usage()
su = usage[usage["step"] == "screen"] if len(usage) else usage
print(f"Screening API calls logged: {len(su)}, ${su['cost_usd'].sum():.3f} in total" if len(su) else "No screening calls logge
ex
```

```
Exclusion list: C:\dev\transcript_intel\data\exclusions.csv (1 row(s))
Folders in data/exclude/: ['01KQ2C53D16CE65DEA612320']
Screening API calls logged: 117, $0.278 in total
```

Out[10]:

	call_id	title	date	action	source	reason	decided_at
0	01KQ2C53D16CE65DEA612320	Team Celebration - Priya's Birthday	07/02/2026	exclude	user	Ignored in the dashboard (keyword "Celebration")	10/29/2026 19:52 UTC

What this shows. Claude judged every loaded call to be business, including internal meetings close to the line (all-hands, executive reviews, a chaos-engineering game day), each with a reason citing the work content. No current title or topic matches the keyword rule.

When this was written, one transcript had been excluded: *Team Celebration – Priya's Birthday* (07/02/2026), ignored in the dashboard before it was processed (it matched "Celebration"). It never reached any table, and its folder is in `data/exclude/`. Any later exclusions appear in the

table above with their source and reason.

5. Call type: derived, not given

The brief describes three call types, but the data has **no call-type field**.

Decision: a deterministic rule (`enrich.derive_call_type`), applied in this order:

1. Title starts with "*Support Case*" → **support**
2. Any attendee outside `aegiscloud.com` → **external**
3. Otherwise → **internal**

A rule is better than an LLM here. The signals are unambiguous, the rule covers 100% of calls, it's free, and anyone can verify it in one line.

In [11]:
```
mix = (calls.groupby(["call_type", "is_synthetic"]).size().unstack(fill_value=0)
       .rename(columns={False: "original", True: "synthetic"}))
mix["total"] = mix.sum(axis=1)
mix
```

Out[11]:

is_synthetic	original	synthetic	total
call_type			
external	43	5	48
internal	30	5	35
support	27	5	32

In [12]:
```
for ct in ["support", "external", "internal"]:
    print(f"\n{ct.upper()}")
    for title in orig.loc[orig["call_type"] == ct, "title"].head(4):
        print("  •", title)
```

```
SUPPORT
  • Support Case #3286 - Vanta Health Systems API Integration Error
  • Support Case #9279 - Summit Trust Billing Inquiry
  • Support Case #3615 - Nimbus Platform Custom Report Request
  • Support Case #6635 - Nova Retail Group SSO Login Failures

EXTERNAL
  • Aegis / Coastal Living Co - Onboarding Kickoff
  • Aegis / Blackridge Investments - Renewal Discussion
  • Aegis / Brightpath Commerce - Detect Module Deployment
  • Aegis / Bridgeport Health - Compliance Review

INTERNAL
  • Detect Team - Sprint Planning
  • Product Sync - Identity Roadmap
  • Comply v2 - Design Review
  • Weekly Engineering Standup
```

Edge cases the rule handles

Escalations don't follow the usual title patterns. Because the rule checks attendee domains rather than parsing titles, a call like *"URGENT: Cobalt Software..."* is still classified as external correctly.

In [13]:
```
odd = orig[~orig["title"].str.match(r"(Support Case|Aegis /)") & (orig["call_type"] != "internal")]
odd[["date", "call_type", "account", "title"]]
```

Out[13]:

	date	call_type	account	title
35	2026-03-10	external	Cobalt	URGENT: Cobalt Software - Aegis Detect Dashboard Down
40	2026-03-12	external	Blackridge	URGENT: Blackridge Investments - Complete Loss of Threat Visibility
45	2026-03-15	external	Northstar	ESCALATION: Northstar Pharma - Detect Outage Impact on Compliance

6. Accounts

Decision. A customer's account is taken from the most common non-AegisCloud email domain on the call (`enrich.extract_account`). Common company-type endings are stripped to get a readable name (`brightpathcommerce.com` → *Brightpath*). This links a customer's **support** tickets to their **account-manager** calls, which no single call shows on its own.

In [14]:
```
acct = (orig.dropna(subset=["account"])
        .pivot_table(index="account", columns="call_type", values="call_id", aggfunc="count", fill_value=0))
acct["total"] = acct.sum(axis=1)
both = acct[(acct.get("support", 0) > 0) & (acct.get("external", 0) > 0)]
print(f"{orig['account'].nunique()} customer accounts in the original data;"
      f" {len(both)} appear in both support and account-manager calls.")
acct.sort_values("total", ascending=False).head(12)
```

```
32 customer accounts in the original data; 18 appear in both support and account-manager calls.
```

Out[14]:

call_type account	external	support	total
Blackridge	3	1	4
Brightpath	2	2	4
Coastal	2	2	4
Silverline	1	3	4
Forge	1	2	3
Ridgeline	1	2	3
Meridian	1	2	3
Crestline	2	1	3
Vanta	1	2	3
Summit	1	2	3
Nimbus	1	1	2
Cobalt	2	0	2

What this shows. Most accounts appear only once or twice, but a core set appear repeatedly across call types. Those accounts are where a joined-up view pays off. For example, a support ticket in March and a renewal call in April for the same customer turn out to be one story. Account health is built on this join in `04_insights`.

Limitation: stripping endings gives short names ("Coastal" for Coastal Living Co). That's fine for grouping, but a production system would use a CRM account ID.

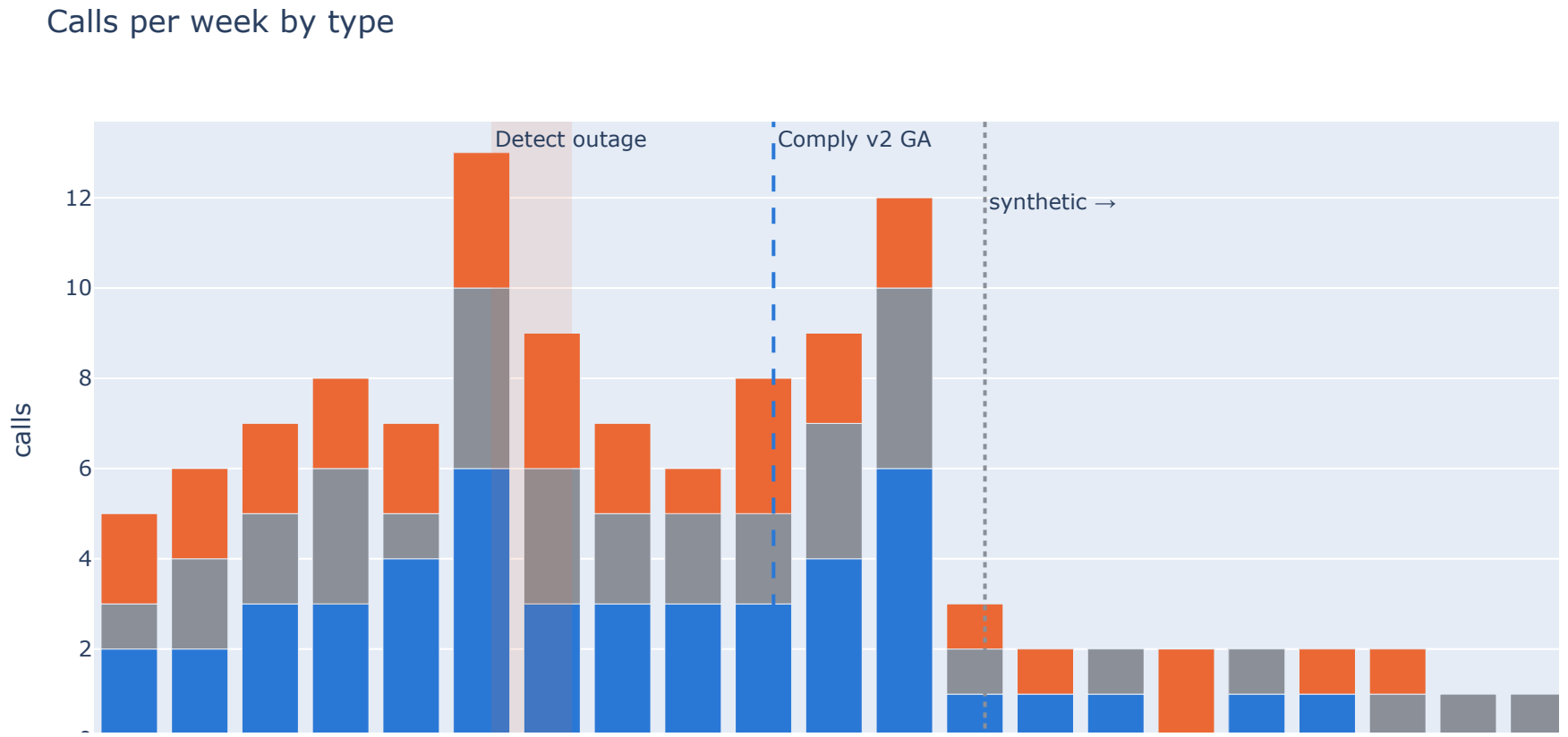
7. Timeline and periods

Decision. Each call is assigned to one of four periods, based on events stated in the calls themselves (`enrich.assign_period`):

Period	Dates	Anchor
Pre-outage	before 10 Mar	baseline
Outage	10–18 Mar	Detect pipeline failure through confirmed full restoration
Recovery	19 Mar – 6 Apr	post-mortems, fixes, customer recovery calls
Post-Comply-launch	from 7 Apr	Comply v2 general availability

These periods are the main lens for the sentiment trends.

In [15]:
```
weekly = (calls.assign(week=pd.to_datetime(calls["date"]).dt.to_period("W").dt.start_time)
          .groupby(["week", "call_type"]).size().reset_index(name="calls"))
fig = px.bar(weekly, x="week", y="calls", color="call_type", color_discrete_map=TYPE_COLORS,
             title="Calls per week by type")
fig.add_vrect(x0=str(OUTAGE_START), x1=str(OUTAGE_END), fillcolor="#eb6834", opacity=0.12,
              line_width=0, annotation_text="Detect outage", annotation_position="top left")
fig.add_vline(x=str(COMPLY_LAUNCH), line_dash="dash", line_color="#2a78d6")
fig.add_vline(x=str(ORIGINAL_END), line_dash="dot", line_color="#8a8f98")
fig.add_annotation(x=str(COMPLY_LAUNCH), y=1, yref="paper", text="Comply v2 GA", showarrow=False, xanchor="left")
fig.add_annotation(x=str(ORIGINAL_END), y=0.9, yref="paper", text="synthetic →", showarrow=False, xanchor="left")
fig
```



In [16]:
```
days = {"Pre-outage": (OUTAGE_START - orig["date"].min()).days,
        "Outage": (OUTAGE_END - OUTAGE_START).days + 1,
        "Recovery": (COMPLY_LAUNCH - OUTAGE_END).days - 1,
        "Post-Comply-launch": (ORIGINAL_END - COMPLY_LAUNCH).days + 1}
per = (orig.groupby("period", observed=False)
          .agg(calls=("call_id", "count"), avg_sentiment=("sentiment_score", "mean"),
               churn_signals=("km_churn_signal", "sum")))
per["days"] = per.index.astype(str).map(days).astype(int)
```

```
per["calls_per_day"] = per["calls"] / per["days"]
per["churn_signals_per_call"] = per["churn_signals"] / per["calls"]
per.round(2)
```

Out[16]:

period	calls	avg_sentiment	churn_signals	days	calls_per_day	churn_signals_per_call
Pre-outage	34	3.78	12	35	0.97	0.35
Outage	17	2.18	18	9	1.89	1.06
Recovery	17	3.45	13	19	0.89	0.76
Post-Comply-launch	32	3.67	18	22	1.45	0.56

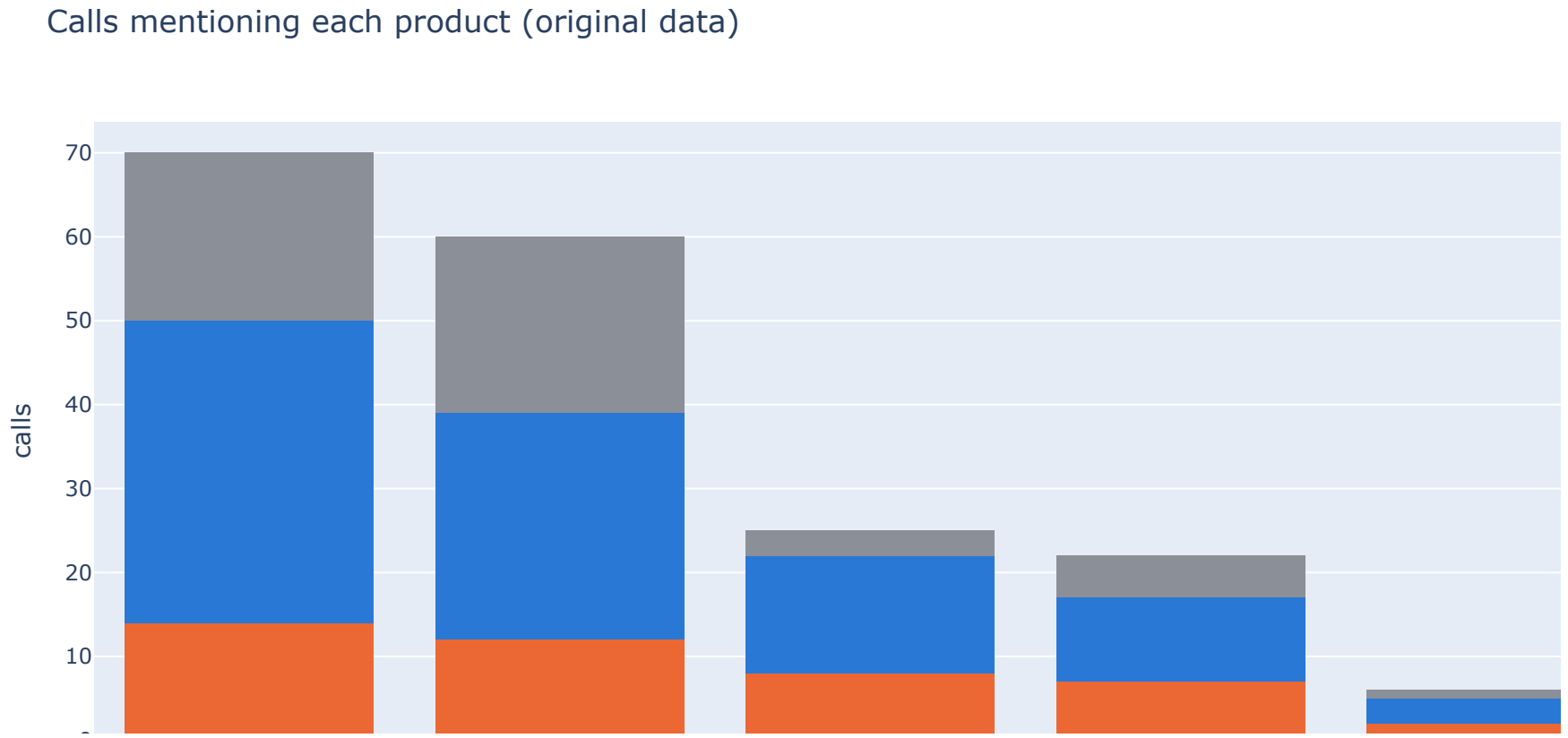
What this shows. Calls per day roughly double during the 9-day outage window, and the increase hits **all three call types at once**: internal war rooms and reviews, customer escalations and support tickets (6 / 6 / 5). One incident pulled engineering, customer success and support into crisis mode at the same time, which is a capacity signal as much as a sentiment one. The outage period also has the lowest average sentiment and the most churn signals per call. That story is explored in `03_sentiment`.

8. Products discussed

Decision. Products are tagged by keyword over each call's title, summary and topics (`enrich.tag_product`). Product names are matched **case-sensitively**, because *protect* and *detect* are also ordinary verbs. Domain terms (backup, SSO, audit…) are case-insensitive. A call can mention several products.

In [17]:

```
prod = orig.explode("products").dropna(subset=["products"])
fig = px.histogram(prod, x="products", color="call_type", color_discrete_map=TYPE_COLORS,
                   barmode="stack", title="Calls mentioning each product (original data)",
                   category_orders={"products": prod["products"].value_counts().index.tolist()})
fig.update_layout(xaxis_title=None, yaxis_title="calls")
fig
```



In [18]:
```
print("Calls with no product tag:", (orig["products"].str.len() == 0).sum())
print("Calls tagged with 2+ products:", (orig["products"].str.len() >= 2).sum())
```

```
Calls with no product tag: 2
Calls tagged with 2+ products: 61
```

9. People and teams

There are no job titles in the data either. `enrich.build_staff_roster` assigns each AegisCloud employee to a team using **role mentions** in the summaries and transcripts first ("*Maria Santos (CSM)*", "*this is Sarah Chen from Aegis support*"). If there are none, it falls back to the kinds of internal meetings they attend. The `basis` column records the evidence, so every assignment can be audited.

In [19]:
```
roster.sort_values(["team", "n_calls"], ascending=[True, False])[["name", "team", "n_calls", "basis"]]
```

Out[19]:

	name	team	n_calls	basis
5	Maria Santos	Customer Success & Sales	15	role mention: CSM at Aegis Cloud Security
4	Lisa Park	Customer Success & Sales	11	role mention: CSM
0	Aisha Johnson	Customer Success & Sales	10	role mention: CSM
2	Diana Reeves	Customer Success & Sales	10	role mention: customer success/sales
6	Rachel Torres	Customer Success & Sales	9	role mention: CSM
1	Daniel Okafor	Customer Success & Sales	8	role mention: CSM
3	James Mitchell	Customer Success & Sales	8	role mention: account manager
12	Sofia Petrov	Engineering	13	role mention: Senior Engineer
8	Mike Romano	Engineering	12	role mention: I'm Mike Romano, Staff Engineer, so I'm sort of Kevin's part
9	Nina Kowalski	Engineering	12	internal meetings (5 engineering, 3 product)
11	Ravi Gupta	Engineering	12	role mention: DevOps
14	Tyler Washington	Engineering	11	role mention: engineering
7	Chris Lee	Engineering	9	internal meetings (6 engineering, 2 product)
13	Tom Bradley	Engineering	8	override
10	Raj Kapoor	Engineering	5	internal meetings (2 engineering, 1 product)
18	Megan Lawson	Product & Strategy	7	role mention: product
17	Jordan Whitfield	Product & Strategy	6	internal meetings (0 engineering, 5 product)
15	Alex Reyes	Product & Strategy	3	internal meetings (0 engineering, 2 product)
16	Hannah Liu	Product & Strategy	3	internal meetings (0 engineering, 2 product)
19	Ananya Sharma	Solutions Engineering	15	role mention: I'm Ananya — I joined the solutions engineering team back in
20	Kevin O'Brien	Solutions Engineering	15	role mention: Solutions Architecture
26	Sarah Chen	Support	12	role mention: this is Sarah Chen from Aegis support, I'll be handling case

	name	team	n_calls	basis
22	David Kim	Support	10	role mention: This is David Kim calling from Aegis Cloud Security support,
23	Elena Vasquez	Support	10	role mention: this is Elena Vasquez from Aegis Cloud Security support, am
24	Marcus Williams	Support	10	role mention: support
21	Brian Cho	Support	8	override
25	Priya Patel	Support	8	role mention: this is Priya Patel from Aegis support — I'm the team lead o

In [20]:
```
side = sents.groupby("speaker_side").agg(sentences=("text", "size"), words=("n_words", "sum"))
side["share_of_words"] = (side["words"] / side["words"].sum()).round(2)
print(f"{roster.shape[0]} AegisCloud staff; "
      f"{sents.loc[sents['speaker_side'] == 'customer', 'speaker_email'].nunique()} customer contacts.")
side
```

```
27 AegisCloud staff; 53 customer contacts.
```

Out[20]:

speaker_side	sentences	words	share_of_words
aegiscloud	3166	102117	0.68
customer	1612	47251	0.32

What this shows. Customer Success and Support carry most of the customer-facing calls, and a handful of people (for example Maria Santos and Ananya Sharma) appear in many more calls than average. AegisCloud staff also do about two-thirds of the talking. That's why `03_sentiment` scores **customer** and **staff** language separately: when staff dominate the conversation, a call-level score mostly measures how AegisCloud sounds, not how the customer feels.

10. First look at the provided topics

Each `summary.json` includes free-text `topics`. These look ready to use, but they aren't.

In [21]:
```
topics = orig.explode("topics")["topics"].str.lower().str.strip()
print(f"{topics.nunique()} unique topic labels across {len(orig)} calls "
      f"({(topics.value_counts() == 1).sum()} are used only once).")
topics.value_counts().head(15).to_frame("calls")
```

```
351 unique topic labels across 100 calls (266 are used only once).
```

Out[21]:

topics	calls
compliance	23
compliance reporting	19
renewal	17
churn risk	14
onboarding	9
outage	8
incident response	8
incident communication	7
product launch	7
feature request	6
platform outage	6
sprint planning	5
identity management	5
product roadmap	5
multi-framework support	5

In [22]:
```
# The same concept appears under many labels
topics[topics.str.contains("outage|incident", regex=True)].value_counts().to_frame("calls")
```

Out[22]:

topics	calls
outage	8
incident response	8
incident communication	7
platform outage	6
service outage	5
outage post-mortem	5
incident review	4
product outage	2
incident communication failure	2
outage follow-up	2
api outage	1
incident post-mortem	1
reliability incident	1
sso outage	1
incident tracking	1
technical incident	1
incident escalation	1
outage remediation	1
outage recovery	1
incident root cause	1
post-incident report	1

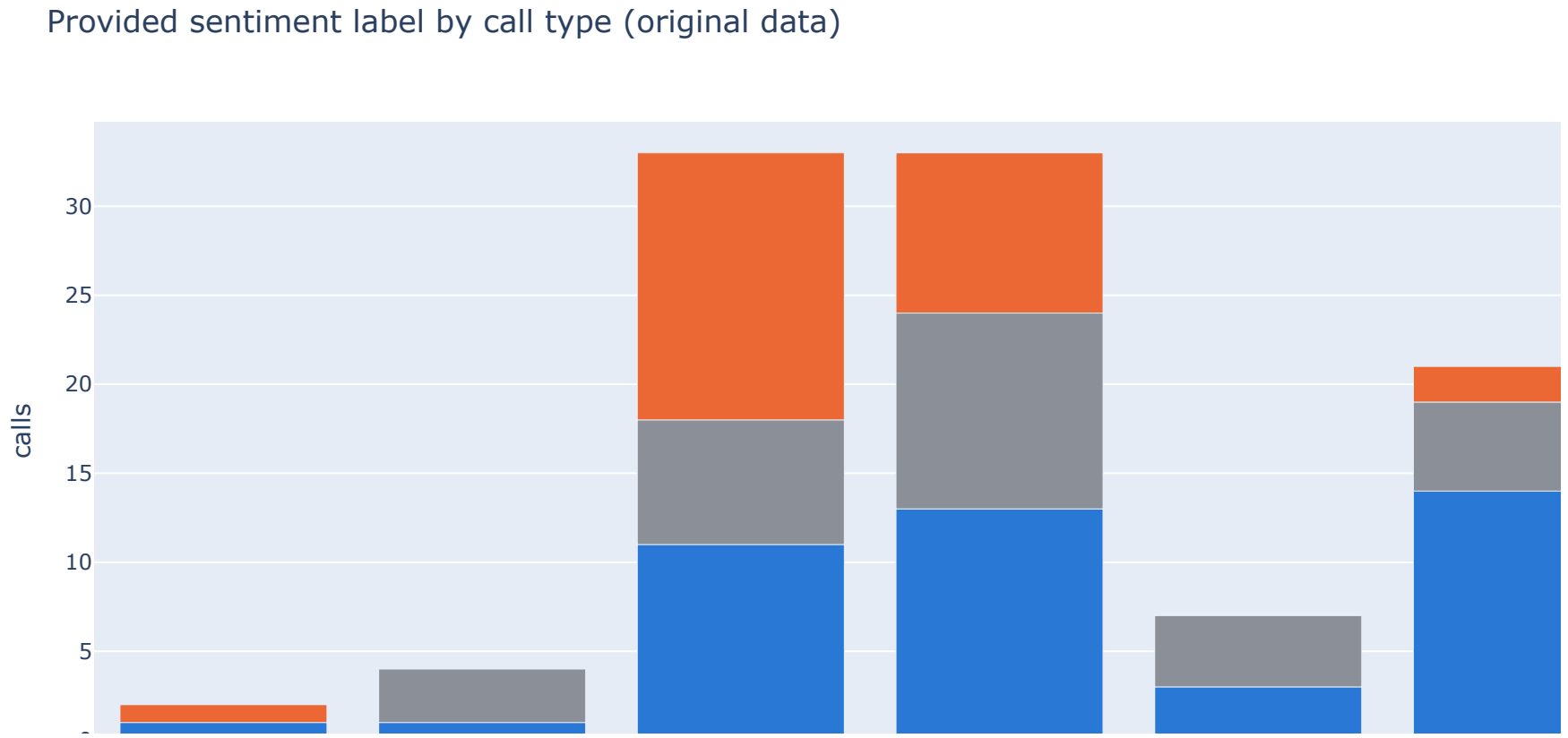
	calls
topics	
incident impact	1
incident communications	1
post-incident review	1
detect outage follow-up	1
incident remediation	1
outage impact	1
platform incident	1
incident risk	1
incident follow-up	1
reliability / outage impact	1

What this shows. The provided topics are too fragmented to group by. One concept, the Detect outage, is spread across many labels ("outage", "platform outage", "service outage", "incident response"...), and most labels are used only once. **This is why Task 1 builds its own taxonomy** (`02_topics`): embedding and clustering discover the structure, then the LLM assigns each call to one of a small set of readable categories.

11. First look at the provided sentiment and key moments

`summary.json` also provides an overall sentiment label, a 1–5 score and tagged **key moments**. These are vendor-generated, so they're treated as one signal to be checked rather than as ground truth (see `03_sentiment`).

In [23]:
```
order = ["very-negative", "negative", "mixed-negative", "mixed-positive", "positive", "very-positive"]
lab = (orig.groupby(["sentiment_label", "call_type"]).size().reset_index(name="calls"))
px.bar(lab, x="sentiment_label", y="calls", color="call_type", color_discrete_map=TYPE_COLORS,
       category_orders={"sentiment_label": order}, title="Provided sentiment label by call type (original data)")
```



In [24]:
```
(orig.pivot_table(index="call_type", columns="month", values="sentiment_score", aggfunc="mean")
     .round(2))
```

Out[24]:

call_type	2026-02	2026-03	2026-04
external	4.26	2.90	4.15
internal	3.76	2.64	3.85
support	2.89	2.73	3.21

In [25]:

```
km = moments.merge(calls[["call_id", "is_synthetic"]], on="call_id")
km = km[~km["is_synthetic"]]
km.pivot_table(index="type", columns="call_type", values="call_id", aggfunc="count", fill_value=0) \
  .assign(total=lambda d: d.sum(axis=1)).sort_values("total", ascending=False)
```

Out[25]:

type	external	internal	support	total
concern	36	23	25	84
positive_pivot	35	26	15	76
churn_signal	26	13	22	61
technical_issue	12	22	20	54
feature_gap	25	15	11	51
action_item	17	17	9	43
praise	14	3	6	23
pricing_offer	7	1	2	10

What this shows. Three early signals, each tested properly later:

1. **March dips for every call type.** External and internal sentiment fall sharply during the outage and recover in April.
2. **Support is the lowest and recovers least.** Support sentiment starts low and stays there, which suggests a backlog or a trust problem that outlasts the incident.

3. **Churn signals are common.** Key moments include dozens of `churn_signal` and `feature_gap` tags. Those become the raw material for account health and the voice-of-customer view in `04_insights`.

12. Synthetic transcripts

The assignment allows generated data as long as it's labelled clearly. Synthetic calls were added to (a) prove the pipeline handles **new input** in the same format, and (b) continue the original storylines into May–July so that trends and account outcomes can be tested.

In [26]:
```
calls.loc[calls["is_synthetic"], ["date", "call_type", "account", "sentiment_label", "title"]] \
     .sort_values("date").reset_index(drop=True)
```

Out[26]:

	date	call_type	account	sentiment_label	title
0	2026-05-06	external	Brightpath	mixed-positive	Aegis / Brightpath Commerce - Detect Technical Session
1	2026-05-08	support	Northstar	mixed-negative	Support Case #4471 - Northstar Pharma Comply v2 HIPAA Report Export Failure
2	2026-05-12	internal	NaN	positive	Detect Chaos Engineering - Game Day Review
3	2026-05-14	external	Summit	mixed-negative	Aegis / Summit Trust - Renewal Decision
4	2026-05-19	support	Crestline	mixed-positive	Support Case #8124 - Crestline Wealth Sync Fix Verification
5	2026-05-21	support	Silverline	negative	Support Case #5530 - Silverline Brands Protect Restore Test Failure
6	2026-05-27	external	Keystone	very-positive	Aegis / Keystone Health - Detect Add-on Decision
7	2026-05-28	internal	NaN	mixed-negative	Customer Success - At-Risk Account Review
8	2026-06-02	support	Frostbyte	mixed-positive	Support Case #6018 - Frostbyte AI Contractor Attribute Mapping Rejected
9	2026-06-04	external	Vanta	positive	Aegis / Vanta Health Systems - Q2 Business Review
10	2026-06-09	internal	NaN	mixed-positive	Pricing & Packaging Review - Comply Usage Tiers
11	2026-06-11	support	Axiom	mixed-negative	Support Case #7311 - Axiom Labs Elastic Seat Invoice Error
12	2026-06-16	internal	NaN	mixed-positive	Protect Scheduler Rewrite - Design Review
13	2026-06-24	internal	NaN	mixed-positive	Q2 Executive Business Review
14	2026-07-01	external	Larkspur	very-positive	Aegis / Larkspur Financial - Onboarding Kickoff

13. Summary

Finding	Implication
Data is structurally clean; every sentence maps to a person	Speaker-level analysis (customer vs staff, talk time) is reliable
Transcripts cover only part of each meeting	Treat time-based metrics as samples; state this as a limitation

Finding	Implication
Non-business calls (birthdays, farewells, socials) would distort every metric	Screened out right after loading (keyword rule + Claude's `is_business`), recorded in `data/exclusions.csv` and moved to `data/exclude/`
Call type, account, product, period and team aren't in the data	Derived with transparent rules in `enrich.py`, each auditable
Provided topics are too fragmented to use directly	Task 1 builds its own taxonomy (`02_topics`)
Provided sentiment shows a March dip and weak support recovery	Task 2 validates and explains it (`03_sentiment`)
Accounts span support and account-manager calls	Enables account health and churn risk (`04_insights`)

Next: `02_topics.ipynb`